

Sistema de Chamados de TI por Prioridade

Descrição do Problema

O gerenciamento eficiente de chamados de suporte técnico é um desafio comum em departamentos de TI. Muitas vezes, solicitações críticas (como queda de servidores) ficam presas atrás de chamados de baixa prioridade (como troca de toners) devido ao uso de filas simples (FIFO).

Este projeto resolve esse problema implementando uma **Fila de Prioridade** que garante que os chamados mais urgentes sejam atendidos primeiro, independentemente da ordem de chegada. Além disso, o sistema conta com um recurso de "desfazer", permitindo reabrir o último chamado resolvido em caso de erro do operador.

Estruturas de Dados Utilizadas

O sistema foi desenvolvido em C puro (C11) utilizando duas estruturas de dados baseadas em nós, sem uso de bibliotecas externas prontas:

1. Árvore Binária de Busca (BST) - Fila de Prioridade

A estrutura principal do sistema é uma Árvore Binária de Busca utilizada para manter a fila de chamados ordenada automaticamente.

- **Por quê:** A BST permite inserir novos chamados e buscar o chamado de maior prioridade (o nó mais à direita da árvore) com eficiência média de $O(\log n)$. É ideal para manter os dados sempre ordenados sem a necessidade de reordenação a cada inserção.
- **Critério de Ordenação:** Os nós são ordenados primeiramente pela prioridade (Baixa, Média, Alta, Crítica) e, em caso de empate, pelo ID do chamado, garantindo a ordem cronológica dentro da mesma prioridade.

2. Pilha (Stack) - Histórico de Operações (Desfazer)

Uma pilha encadeada foi implementada para armazenar o histórico de resoluções.

- **Por quê:** O recurso de "desfazer a última operação" segue naturalmente o conceito LIFO (Last In, First Out). Quando um chamado é resolvido, ele é removido da BST e empilhado (Push). Se o usuário desejar desfazer, o chamado é desempilhado (Pop) e reinserido na BST.

Formato do Arquivo de Persistência

Os dados são persistidos em um arquivo **CSV (Comma-Separated Values)** nomeado `chamados.csv`.

- **Por quê:** O formato CSV foi escolhido por ser leve, legível por humanos e facilmente integrável com outras ferramentas (como Excel, bancos de dados, etc.).
- **Estrutura:** `id,descricao,prioridade,data_criacao`
- **Exemplo de conteúdo:**

Plain Text

```
id,descricao,prioridade,data_criacao
1,Servidor crítico offline,Crítica,2024-05-20 14:30:00
2,Troca de toner,Baixa,2024-05-20 15:00:00
```

Limitações Conhecidas

1. **Balanceamento da Árvore:** A BST implementada não é auto-balanceável (como AVL ou Red-Black). Em um cenário extremo de inserções sequenciais estritas, ela pode se degradar para uma lista encadeada, tornando as operações $O(n)$.
2. **Concorrência:** O sistema não possui controle de concorrência (locks/mutexes), sendo projetado para uso single-user local.
3. **Tamanho da Descrição:** A descrição dos chamados é estaticamente limitada a 255 caracteres para simplificar a alocação de memória e o parse do CSV.
4. **Capacidade Máxima:** Há um limite de segurança de 1000 chamados simultâneos na árvore para evitar estouro de memória em ambientes restritos.

Compilação e Execução

O projeto inclui um `Makefile` para facilitar a compilação.

Requisitos: GCC (ou outro compilador C compatível) e Make.

Bash

```
# Compilar o projeto
make all

# Executar
make run
```

```
# Limpar arquivos gerados  
make clean
```

Estrutura de Arquivos

- `main.c` : Ponto de entrada e menu interativo.
- `arvore.c` : Implementação da Árvore Binária de Busca.
- `pilha.c` : Implementação da Pilha de histórico.
- `persistencia.c` : Leitura e escrita do arquivo CSV.
- `utils.c` : Funções auxiliares de formatação e conversão.
- `chamados.h` : Definições de tipos e protótipos de funções.